Processus d'Entraînement & Paramètres Finaux — T-AIA-911

Projet : Extraction d'intentions de voyage
Date : Février 2026

---

1. Vue Générale du Pipeline d'Entraînement

```
Données Brutes (phrases)
    │
    ▼
Générateur Synthétique (dataset/generators/)
    │  • 50 000+ phrases VALID/INVALID pour Classifier
    │  • 40 000+ phrases annotées IOB pour NER
    ▼
Preprocessing & Split (Train / Val / Test)
    │
    ├──────────────────────────────────────────┐
    ▼                                          ▼
CLASSIFIER Training                        NER Training
(CamemBERT Séquence)                   (CamemBERT Token)
    │                                          │
    ▼                                          ▼
Modèle Final Classifier                  Modèle Final NER
(validity_classifier_camembert_v1)       (ner_camembert_v1)
```

---

2. Construction du Dataset

2.1 Classifier — Dataset de Classification

Objectif : Classer chaque phrase en VALID (demande de trajet) ou INVALID.

Volume : ~50 000 phrases générées synthétiquement

Distribution des classes :

Classe	%	Description
INVALID	70%	Phrases sans intention de trajet
VALID	30%	Phrases avec départ ET arrivée explicites

Types de phrases VALID générées :

- Formes déclaratives : *"Je vais de Paris à Lyon"*
- Formes interrogatives : *"Comment aller de Lille à Marseille ?"*
- Formes familières/SMS : *"jveux un train pariis bordeaux stp"*
- Avec fautes d'orthographe : *"Je parte de Toulousse vers Bordeauxe"*

Types de phrases INVALID générées :

- Questions générales : *"Quelle est la météo à Paris ?"*
- Phrases avec une seule ville : *"Je vais à Paris"*
- Texte aléatoire : *"asdadadasd"*, *"pizza quatre fromages"*
- Descriptions de lieux : *"Paris est une belle ville"*

Split train/val/test :

Set	%	Stratification
Train	70%	Oui (par classe)
Validation	15%	Oui
Test	15%	Oui
Random seed	42	Fixe pour reproductibilité

2.2 NER — Dataset d'Extraction d'Entités

Objectif : Annoter chaque token en O / B-DEPARTURE / I-DEPARTURE / B-ARRIVAL / I-ARRIVAL.

Volume : ~40 000 phrases annotées au format JSONL

Format d'annotation :

```
{
  "text": "Je veux aller de Paris à Lyon",
  "entities": [
    [17, 22, "DEPARTURE"],
    [25, 29, "ARRIVAL"]
  ]
}
```

Évolution du dataset (itérations successives) :

Version	Ajout Principal	Objectif
v1 (initial)	Phrases basiques de trajet	Baseline
v2 (+enrichissement)	Fautes d'orthographe, SMS	Robustesse
v3 (+corrections générateur)	Gestion séparateur " - " entre villes	Réduire FP tirets
v4 (+post-processing tirets)	Filtrage gares (post-processing)	Réduire FP
v5 (final)	**Exemples négatifs 12%** (prénoms, pays, villes hors contexte)	**Résoudre plateau précision**

Split train/val/test NER :

Set	%
Train	70%
Validation	15%
Test	15%

---

3. Processus d'Entraînement — Classifier

3.1 Baseline (TF-IDF + Régression Logistique)

Avant le fine-tuning CamemBERT, un modèle baseline a été entraîné pour établir une référence :

```
TfidfVectorizer(ngram_range=(1, 2), max_features=50000, min_df=2, max_df=0.95)
LogisticRegression(C=1.0, solver='lbfgs', class_weight='balanced')
```

Résultats baseline :

Métrique	Score
Accuracy	85.5%
Précision	82.1%
Rappel	86.3%

3.2 Fine-tuning CamemBERT

Modèle de base : `camembert-base` (HuggingFace)

Configuration d'entraînement finale :

```
TrainingArguments(
    num_train_epochs        = 3,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size  = 16,
    learning_rate           = 2e-05,
    weight_decay            = 0.01,
    warmup_steps            = 500,
    eval_strategy           = "steps",
    eval_steps              = 100,
    save_steps              = 200,
    load_best_model_at_end  = True,
    metric_for_best_model   = "f1",
    early_stopping_patience = 2
)
```

Tokenization :

```
CamembertTokenizer(
    max_length  = 128,
    padding     = 'max_length',
    truncation  = True
)
```

---

4. Processus d'Entraînement — NER

4.1 Modèle Initial

Même architecture `camembert-base` avec tête de classification par token.

Problème identifié dès le début : Précision faible (~~74%) malgré un bon rappel (~~90%).
→ Le modèle annotat trop de tokens comme des villes (faux positifs).

4.2 Itérations d'Amélioration

Itération 1 : Enrichissement Dataset

- Ajout de patterns avec prénoms similaires à des villes (Florence, Nancy, Paris comme prénom)

- Ajout de villes mentionnées hors contexte de trajet

Itération 2 : Corrections du Générateur

- Correction du pattern "Ville - Ville" (tirets avec espaces = séparateur, pas partie d'entité)
- Résultat : réduction des FP sur tirets (16% → ~5%)

Itération 3 : Post-Processing

- Ajout d'un filtrage des entités < 2 caractères
- Validation optionnelle par liste `gares-francaises.json`

Itération 4 (Décisive) : Exemples Négatifs 12%

- Injection de 12% d'exemples où des noms propres similaires à des villes n'étaient PAS annotés
- Ajout de pays (Tunisie, Turquie) non annotés
- **Résultat : précision +17 points** (75% → 92%)

Configuration finale identique au Classifier (mêmes hyperparamètres).

---

5. Paramètres Finaux des Modèles

5.1 Classifier Final

Modèle : `validity_classifier_camembert_20260109_153726_v1`

Paramètre	Valeur
Base model	camembert-base
max_length	128
batch_size	16
learning_rate	2e-05
weight_decay	0.01
num_epochs	3
num_labels	2 (VALID, INVALID)
metric_best_model	f1

Métriques test set finales :

Métrique	Score
Accuracy	**99.42%**
Précision	**99.36%**
Rappel	**99.81%**
F1-Score	**99.58%**
AUC-ROC	**99.98%**

5.2 NER Final

Modèle : `ner_camembert_20260116_171357_v1`

Paramètre	Valeur

Base model	camembert-base
max_length	128
batch_size	16
learning_rate	2e-05
weight_decay	0.01
num_epochs	3
num_labels	5 (O, B-DEP, I-DEP, B-ARR, I-ARR)
metric_best_model	f1
eval_steps	100
early_stopping_patience	2

Métriques test set finales :

Métrique	Global	DEPARTURE	ARRIVAL
Accuracy	**98.24%**	—	—
Précision	**94.82%**	95.19%	94.46%
Rappel	**95.43%**	95.33%	95.52%
F1-Score	**95.12%**	95.26%	94.99%

---

6. Courbe d'Apprentissage NER (Modèle Final)

Extrait de l'historique d'entraînement (F1 sur validation) :

Step	Époque	F1-Val	Précision-Val	Loss-Val
100	0.11	34.1%	34.7%	0.656
300	0.32	87.0%	84.5%	0.284
500	0.53	91.7%	90.6%	0.192
800	0.84	94.3%	93.8%	0.135
1000	1.05	94.7%	94.0%	0.113
1300	1.37	**95.4%**	**95.0%**	0.099
1500	1.58	95.5%	95.1%	0.091
1700	1.79	95.3%	94.7%	0.084

→ Le modèle atteint sa meilleure performance autour du step 1400-1500 (époque ~1.5).
→ Early stopping activé après 2 epochs sans amélioration sur F1.

---

7. Résumé Comparatif des Expériences NER

Modèle	Précision	Rappel	F1	Note
Initial (09/01)	67.3%	88.0%	76.3%	Baseline NER
+enrichissement (12/01)	74.4%	90.6%	81.7%	Dataset v2
+corrections générateur (15/01)	75.7%	92.1%	83.1%	Plateau précision
+post-processing tirets (15/01)	79.5%	91.8%	85.2%	Réduction FP
+exemples négatifs 12% (16/01)	**94.8%**	**95.4%**	**95.1%**	**Percée décisive**

Document généré le 27/02/2026